OSTRAVSKÁ UNIVERZITA
ÚSTAV PRO VÝZKUM A APLIKACE FUZZY MODELOVÁNÍ

7LLMA · TAHÁK PRO STUDENTY

# Volání modelů přes API

OpenAI · Anthropic · OpenRouter · Microsoft Foundry · streaming · historie · chyby

k decku 7LLMA-04-volani-v1_0 · stav 8. 10. 2026

## Čtyři API vedle sebe

| | OpenAI | Anthropic | OpenRouter | Microsoft Foundry |
|---|---|---|---|---|
| **Adresa** | `api.openai.com/v1` | `api.anthropic.com/v1` | `openrouter.ai/api/v1` | `RESOURCE.openai.azure.com/openai/v1` |
| **Endpoint** | `/responses` (nové), `/chat/completions` | `/messages` | `/chat/completions` | `/responses`, `/chat/completions` |
| **Autentizace** | `Authorization: Bearer` | `x-api-key` + `anthropic-version` | `Authorization: Bearer` | `api-key` nebo Entra ID (Bearer) |
| **Model** | `gpt-6.1-sol` | `claude-opus-5-5` | `vendor/model` | název nasazení |
| **System prompt** | `instructions` | `system` (mimo zprávy) | zpráva `role: system` | `instructions` |
| **Python** | `openai` | `anthropic` | `openai` + `base_url` | `openai` + `base_url` |
| **Stream** | `stream=True` → události | `messages.stream()` → události | `stream=True` → chunky | `stream=True` → události |

Tři ze čtyř používají knihovnu `openai`; Anthropic má vlastní tvar požadavku i odpovědi. Id modelů: stav k 8. 10. 2026, před použitím ověřit.

## Anatomie jednoho volání

| Část | Co to je |
|---|---|
| **Endpoint** | HTTPS POST na URL poskytovatele; JSON tam, JSON zpět; SDK je jen obal |
| **Klíč** | tajný řetězec v hlavičce nebo cloudová identita; proměnná prostředí, ne kód |
| **Model** | u vendora přesné id, u OpenRouteru `vendor/model`, na Foundry název nasazení |
| **Zprávy** | system + user + assistant; API je bezstavové |
| **Odpověď** | text + důvod konce (hotovo, uříznuto, odmítnuto, nástroj) – vždy zkontrolovat |
| **Usage** | tokeny vstup + výstup (vč. přemýšlení) = cena; logovat |

Z localhostu do cloudu se mění tři věci: `base_url`, klíč a id modelu. S nimi cena, limity, smlouva a kde leží data.

## Jednoduché volání

```
# OpenAI – Responses API
from openai import OpenAI
client = OpenAI()            # OPENAI_API_KEY
r = client.responses.create(
    model="gpt-6.1-sol",
    instructions="Odpovídej česky.",
    input="Co je inference?")
print(r.output_text, r.status,
      r.usage.input_tokens, r.usage.output_tokens)
```

```
# Anthropic – Messages API
import anthropic
client = anthropic.Anthropic()  # ANTHROPIC_API_KEY
msg = client.messages.create(
    model="claude-opus-5-5",
    max_tokens=2000,              # povinné
    system="Odpovídej česky.",    # mimo zprávy
    messages=[{"role": "user",
               "content": "Co je inference?"}])
for b in msg.content:
    if b.type == "text": print(b.text)
print(msg.stop_reason, msg.usage.output_tokens)
```

```
# OpenRouter – Chat Completions, knihovna openai
client = OpenAI(
    base_url="https://openrouter.ai/api/v1",
    api_key=os.environ["OPENROUTER_API_KEY"])
r = client.chat.completions.create(
    model="anthropic/claude-haiku-4.5",
    messages=[
      {"role": "system", "content": "Česky."},
      {"role": "user", "content": "Co je token?"}])
print(r.choices[0].message.content, r.model)
```

```
# Foundry – tentýž klient, jiná adresa
client = OpenAI(
    base_url="https://MUJ-RESOURCE"
             ".openai.azure.com/openai/v1/",
    api_key=os.environ["AZURE_OPENAI_API_KEY"])
# bez klíče (Entra ID, az login):
#   api_key=get_bearer_token_provider(
#       DefaultAzureCredential(),
#       "https://ai.azure.com/.default")
r = client.responses.create(
    model="moje-nasazeni",   # název nasazení!
    input="Co je inference?")
```

- **Foundry:** žádný `api-version`; token Entra ID se obnovuje sám (`pip install azure-identity`). Nasazení typu „Data Zone EU“ drží výpočet v EU.
- **Claude na Foundry:** `anthropic.AnthropicFoundry(resource=…)`, dál stejné `messages.create`.

## Anthropic: co je jinak

- Klíč v `x-api-key` + povinná hlavička `anthropic-version: 2023-06-01`.
- `max_tokens` je povinné; Opus 5.5 vždy přemýšlí a přemýšlení se do něj počítá – nedávat těsně.
- `system` stojí mimo zprávy; v `messages` jen user a assistant.
- Opus 5.5 nepřijímá `temperature`; hloubku řídí `output_config={"effort": …}`.
- `stop_reason == "refusal"` → content může být prázdný; `stop_details` řekne proč.

## Kde je co v odpovědi

| Hledám | Responses (OpenAI, Foundry) | Chat Completions (OpenRouter, Ollama…) | Messages (Anthropic) |
|---|---|---|---|
| **Text** | `r.output_text` | `r.choices[0].message.content` | `msg.content[i].text` (bloky) |
| **Proč skončil** | `r.status`, `r.incomplete_details` | `choices[0].finish_reason`: stop, length… | `msg.stop_reason`: end_turn, max_tokens, refusal… |
| **Tokeny vstup** | `r.usage.input_tokens` | `r.usage.prompt_tokens` | `msg.usage.input_tokens` |
| **Tokeny výstup** | `r.usage.output_tokens` | `r.usage.completion_tokens` | `msg.usage.output_tokens` |
| **Limit délky** | `max_output_tokens` | `max_tokens` / `max_completion_tokens` | `max_tokens` (povinné) |
| **System prompt** | `instructions=` | zpráva `{"role": "system"}` | `system=` |

**POZOR** Prázdný nebo useknutý text bez chyby? Limit délky i odmítnutí vrací HTTP 200 – číst důvod konce.

## Streaming: proč a co teče po drátě

- **První slovo hned:** celkový čas stejný, uživatel čte od první vteřiny.
- **Dlouhé odpovědi:** bez streamu hrozí timeout; reasoning modely odpovídají i minuty.
- **Stejné tokeny, stejná cena:** stream je jen způsob doručení.
- `"stream": true` → HTTP odpověď zůstane otevřená, typ `text/event-stream` (Server-Sent Events). Události = řádky `data: {json}` oddělené prázdným řádkem; řádek začínající dvojtečkou je komentář.

```
: OPENROUTER PROCESSING
data: {"choices":[{"delta":{"content":"Infer"}}]}
data: {"choices":[{"delta":{"content":"ence"}}]}
data: {"choices":[{"delta":{},
  "finish_reason":"stop"}],"usage":{…}}
data: [DONE]
```

## Stream: dva tvary

**Chunky s delta** – Chat Completions (OpenRouter, Ollama, vLLM, OpenAI, Foundry):

```
stream = client.chat.completions.create(
    model=MODEL, messages=msgs, stream=True,
    stream_options={"include_usage": True})
for chunk in stream:
    if getattr(chunk, "error", None):
        print("CHYBA:", chunk.error); break
    if chunk.choices:
        kus = chunk.choices[0].delta.content
        if kus: print(kus, end="", flush=True)
    if chunk.usage: usage = chunk.usage  # poslední
```

**Pojmenované události** – Responses (OpenAI, Foundry):

```
for event in client.responses.create(
        model=MODEL, input=q, stream=True):
    if event.type == "response.output_text.delta":
        print(event.delta, end="", flush=True)
    elif event.type == "response.completed":
        usage = event.response.usage
    elif event.type == "error":
        print("CHYBA:", event.message)
```

**Anthropic** – pomocník SDK:

```
with client.messages.stream(model=MODEL,
        max_tokens=4000, messages=msgs) as s:
    for text in s.text_stream:
        print(text, end="", flush=True)
    final = s.get_final_message()  # stop, usage
```

- Anthropic po drátě: `message_start` → `content_block_start` → `content_block_delta` (×n) → `content_block_stop` → `message_delta` (stop_reason, usage) → `message_stop`; kdykoli `ping` a `error`. Neznámou událost přeskočit, ne spadnout.
- Usage v posledním chunku: OpenRouter posílá vždy, jinde `stream_options={"include_usage": True}`.
- `flush=True`, jinak Python výstup zadrží a „stream“ se vypíše naráz.

## Pasti streamingu

- **Tokeny až na konci:** usage je v poslední události; kdo stream utne dřív, nemá co logovat.
- **Chyba po HTTP 200:** hlavička už odešla; chyba přijde jako událost – kontrolovat v cyklu, ne jen `try` kolem volání.
- **Dlouho nic:** reasoning model přemýšlí; `ping` a komentáře drží spojení; ticho ≠ konec.
- **Buffering:** nginx, firemní proxy i `curl` bez `-N` pustí vše naráz; `proxy_buffering off`, v Pythonu flush.
- **Uživatel zavřel okno:** zavřít spojení, jinak model generuje a účtuje dál (OpenRouter zastaví účtování jen u části poskytovatelů).
- **Strukturovaný výstup:** půlka JSONu není JSON; parsovat až po dokončení, nebo streamovat jen text pro člověka.

## Historie konverzace

```
historie = [{"role": "system",
             "content": "Odpovídej česky."}]
while (dotaz := input("vy> ")):
    historie.append({"role": "user",
                     "content": dotaz})
    odpoved = ""
    for chunk in client.chat.completions.create(
            model=MODEL, messages=historie,
            stream=True):
        kus = (chunk.choices[0].delta.content
               if chunk.choices else None) or ""
        odpoved += kus
        print(kus, end="", flush=True)
    historie.append({"role": "assistant",
                     "content": odpoved})
```

- **API si nic nepamatuje:** „paměť“ chatu je seznam ve vašem programu. Uložit i odpověď, jinak model zapomene, co řekl.
- **Historie roste:** každé kolo platí všechny předchozí zprávy znovu; dlouhé chaty zkracovat nebo shrnovat.
- **Stav na serveru:** OpenAI Responses `previous_response_id`; pohodlné, ale váže na jednoho poskytovatele.

## Jedna funkce, víc backendů

| Backend | base_url | Model |
|---|---|---|
| **openai** | `api.openai.com/v1` | `gpt-6.1-sol` |
| **openrouter** | `openrouter.ai/api/v1` | `vendor/model` |
| **foundry** | `…/openai/v1/` | název nasazení |
| **ollama** | `localhost:11434/v1` | `gemma4:e4b` |
| **anthropic** | – (Messages) | `claude-opus-5-5` |

- Dva tvary stačí: první čtyři mluví OpenAI Chat Completions, Anthropic má Messages. `stream(backend, prompt)` vrací kousky textu; aplikace neví, kdo odpovídá.
- Konfigurace místo kódu: výměna backendu = jeden řádek. Hotová řešení: LiteLLM, OpenRouter; vlastní vrstva má 30 řádků.

## Chyby, opakování, timeout

```
client = OpenAI(max_retries=4,  # výchozí 2
                timeout=30)     # výchozí 600 s
try:
    r = client.responses.create(...)
except openai.RateLimitError:
    ...                         # 429 i po opakování
except openai.APIStatusError as e:
    print(e.status_code, e.request_id)
except openai.APIConnectionError:
    ...                         # síť, DNS, timeout
# anthropic: stejně pojmenované výjimky
```

| Kód | Význam | Co dělat |
|---|---|---|
| **400** | špatný požadavek | opravit, neopakovat |
| **401 / 403** | klíč, práva | opravit |
| **404** | model / nasazení | opravit id |
| **429** | limit | zpomalit, fronta; SDK opakuje |
| **5xx / 529** | přetížení | opakovat s čekáním |

- Dlouhé odpovědi streamovat, ne prodlužovat timeout.
- Logovat model, tokeny, latenci, request id; ne prompty s osobními údaji, nikdy klíče.
- Limit útraty v konzoli poskytovatele; zvláštní klíč pro každou aplikaci.

## Stream až do prohlížeče

```
@app.get("/chat")
async def chat(q: str):
    return StreamingResponse(udalosti(q),
        media_type="text/event-stream")

async def udalosti(q):
    stream = await client.chat.completions.create(
        model=MODEL, stream=True,
        messages=[{"role": "user", "content": q}])
    async for chunk in stream:
        if chunk.choices:
            kus = chunk.choices[0].delta.content
            if kus:
                yield f"data: {json.dumps(kus)}\n\n"
    yield "data: [DONE]\n\n"
```

- **Prohlížeč:** `es = new EventSource("/chat?q=…")`, `es.onmessage` připisuje text, jak přichází; tři řádky JavaScriptu.
- **Klíč nesmí do prohlížeče:** JavaScript na stránce si přečte kdokoli; model volá váš server.
- Chyba uprostřed: hlavička 200 už odešla → poslat vlastní událost `event: error` a ukončit.
- Za nginxem: `proxy_buffering off` nebo hlavička `X-Accel-Buffering: no`.

## Na zapamatování

TIP **Volání = HTTPS POST:** adresa, klíč, model, zprávy → text, důvod konce, tokeny. **Tři ze čtyř** jdou přes knihovnu `openai` a `base_url`. **Stream** = stejná cena, lepší dojem; usage a chyby až v posledních událostech. **Historii držíte vy. Klíč na server**, nikdy do prohlížeče ani gitu.

Zdroj: přednáška Volání modelů přes API (7LLMA-04-volani-v1_0.pptx), stav k 8. 10. 2026.